

One Bit

UM-NATOS-030

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-030	1.0 · 2026-08-18	**Fixed and confirmed on hardware**	Internal

1. Abstract

The 3D view fault is closed. It was one bit.

`SPI_DMA_OUT_LINK_REG` on the ESP32 carries the descriptor address in bits 19:0, `OUTLINK_STOP` at 28, `OUTLINK_START` at 29, and `OUTLINK_RESTART` at 30. This driver defined `DMA_OUTLINK_START` as `(1u << 30)`.

Every DMA transfer nat-os has ever issued to the panel told the engine to *resume the existing descriptor chain from wherever it left off*, rather than to *begin the descriptor just written*. The transfer completes. `SPI_USR` clears. No timeout fires. The byte counters advance. The boot self-test reports `dma=320/0`. And the pixels land progressively displaced.

Eleven theories were investigated and eliminated before this was found (UM-NATOS-029 §4). All eleven were downstream of it, and several were eliminated *correctly* — on a system that had already fallen back to the FIFO path, where the bug genuinely was not present.

The fix is one character. The report is about why it took a day.

1.1 The change

In `kernel/display.c`:

```
/* before */
#define DMA_OUTLINK_START (1u << 30)

/* after */
#define DMA_OUTLINK_STOP (1u << 28)
#define DMA_OUTLINK_START (1u << 29)
#define DMA_OUTLINK_RESTART (1u << 30)
```

Nothing else about the transfer changed. `spi2_dma_tx()` still writes the same descriptor and still starts the link the same way:

```
GPIO_REG(SPI2_DMA_OUT_LINK) = ((uint32_t)&g_desc & 0xFFFFFu) | DMA_OUTLINK_START;
```

All three constants are defined rather than only the one in use, so the next reader sees the adjacency that caused this and does not have to go back to the technical reference manual to check which neighbour they are looking at.

1.2 Scope: this was never only the 3D view

Confirmed on hardware after the fix: **other graphical glitches across the UI disappeared at the same time**, and had been attributed to unrelated causes.

That follows directly. `spi_tx()` sends anything over 64 bytes by DMA, so the defect touched every transfer wider than 32 pixels — full-screen clears, the colour strip, launcher repaints, `display_text()` on longer strings, the panic screen. Only small fills and command bytes escaped, because they go out through the W registers on the FIFO path.

The 3D view was simply the worst-affected consumer: 224 DMA transfers per frame inside a single window, more than everything else in the system combined. It was not a renderer bug that happened to look like a display bug. It was a display bug that the renderer exercised hardest.

2. Why it was invisible

The peripheral reports success because, from its point of view, it succeeded. `OUTLINK_RESTART` is a legitimate command; the engine performs it and retires the transfer. Everything the kernel can observe agrees:

instrument	reading	truth
<code>SPI_USR</code> self-clearing	transfer complete	it was
<code>display_dma_transfers()</code>	110,507 and climbing	it was
<code>display_dma_timeouts()</code>	0	correct
<code>display_bytes_written()</code>	matches expectation	it does
boot self-test <code>dma=320/0</code>	healthy	at boot, with nothing to preempt
the panel	wrong	—

Six instruments in agreement, one disagreeing, and the one disagreeing was the only one not attached to the machine. This is UM-NATOS-028 §8's theme carried to its conclusion: **the kernel could not see the only thing that was wrong**.

3. The causal chain, in full

The FIFO path never touches `SPI_DMA_OUT_LINK_REG`. That single fact explains every observation recorded over two sessions.

The view garbles from boot. DMA is alive; the blit is displaced.

It heals ten to sixty seconds later. `spi2_dma_tx()` bounded its wait on wall clock at 2,000,000 cycles (~25 ms), which a preempted display task trips on a healthy engine. The trip sets `g_dma_ok = 0` permanently, everything falls back to the FIFO, and the picture becomes correct. **The heal and the spurious timeout are the same event**, at about twelve seconds.

gfxrogue "repairs" it. Its clipped fill is 180 px — 360 bytes a row, over `spi_tx()`'s 64-byte threshold — so it issues DMA transfers, adds opportunities for the display task to be descheduled mid-wait, and brings the timeout forward. It was never repairing anything. It was killing the DMA engine faster.

hog draw repairs it. Same mechanism, same width, and it stopped working the moment the timeout bound was raised — because the thing it was provoking could no longer happen.

The draw application never repairs it. Its block is 20 px: 40 bytes a row, below the threshold, entirely on the FIFO, incapable of provoking a DMA timeout. This was the observation that fitted no theory for hours. It is the one that confirms this one.

"Leave it five minutes and it opens fine." The timeout arrives on its own.

Raising the timeout bound made the device worse. The bound was a genuine defect and the reasoning for fixing it was sound in isolation. It was also the only thing keeping the display usable, and removing it left the view broken indefinitely — including for a single static test pattern.

Every one of those was reported as a separate mystery. There was one bug.

4. What actually found it

Not reasoning. Reasoning produced eleven wrong answers.

4.1 fbdump — make the buffer visible

`fbsum` reported a checksum; `fbhash` reported a better checksum. Neither can say whether a frame is a corridor or noise, and a frozen camera stably rendering a *wrong* scene gives a perfectly constant hash. MISO is documented as wired on GPIO12 but reads all zeros (§6.1), so the panel cannot be read back.

`fbdump` sends the framebuffer out over the UART as hex; `grab_fb.py` reconstructs it as a PNG. The first dump taken while the panel was garbled showed a **pristine corridor** — correct perspective, correct shading, the close button correctly stamped.

That single image cut the search from "the system" to "between the buffer and the glass".

4.2 fbpattern — remove everything else

Eight flat colour bands, blitted through the identical `display_blit()` with the renderer frozen. **A single static image came out corrupt**, which removed the raycaster, the scheduler, the applications, the touch task and all concurrency from suspicion at once.

After that, `spi2_dma_tx()` is thirty lines and the register map is a short read.

5. Instruments that lied — including the new ones

UM-NATOS-028 §8 counted seven. UM-NATOS-029 §7 added five. This session added three more, and two of them were built *to end this class of problem*.

5.1 fbdump manufactured the fault it was built to photograph

The first dump ran without holding the console. The reporter's periodic status line landed in the middle of the hex, shifting rows; the host parser silently discarded 61 malformed rows and reassembled the survivors as if contiguous. The resulting image was a convincing rainbow-streaked mess, and it was reported as proof of a render bug.

It was an artifact. With `console_lock()` held across the dump and each row carrying its own index, the same buffer is pristine. Fixed by locking, by indexing rows so position is *read* rather than inferred from arrival order, and by making the host refuse to interpret an incomplete dump.

5.2 A fixed capture window truncated the dump at row 49, twice

Silently. Now the host reads until the `FBEND` terminator and reports `rows received N/112, malformed M, missing [...]`.

5.3 A constant hash was called decisive and was not

`camfreeze + fbhash` was presented as settling render-versus-transport. It does not: a frozen camera stably rendering a wrong scene produces a constant hash too. The test was sound; the claim made for it was not.

5.4 Carried forward from UM-NATOS-029 §7.2

The timeout bound was raised, `dmastat` was read as `timeouts=0`, and that was taken as evidence the raise had been unnecessary — circular, since the raise is why it read zero. **A guard can only be shown unnecessary by measuring it in its absence.**

6. Standing rules earned here

1. **A counter cannot see a picture.** Six instruments agreed the display was working. Where the output is visual, get the output out of the machine and look at it; do not accept a proxy.
 2. **When a diagnostic and a subsystem share a resource, the diagnostic will corrupt the evidence.** Hold the lock, or measure something else.
 3. **Read positions, never infer them from arrival order.** Indexed rows caught what a silent parser had hidden twice.
 4. **A tool must refuse to interpret incomplete input.** `!! DUMP INCOMPLETE` is worth more than a plausible image.
 5. **Before fixing a defect, find out what depends on it.** The timeout bound was genuinely wrong and had become load-bearing.
 6. **One-bit register constants deserve the datasheet, not recall.** `START` and `RESTART` are adjacent, and both produce a transfer that completes.
-

7. Also fixed, and still open

Fixed alongside:

- All three DMA resets per transfer (`OUT_RST` , `AHBM_FIFO_RST` , `AHBM_RST`), not just the outbound channel. The driver alternates transports — `set_window()` sends command bytes through the W registers, pixels go by DMA — and they share the peripheral's AHB master FIFO.
- The timeout bound stays at 40,000,000 (~500 ms). It was always the right change; it is now safe to keep, because the thing it was masking is gone.

Still open:

- **MISO reads all zeros.** `UM-NATOS-015` records GPIO12 as "wired but unused"; `panelid` gets `00 00 00 00 00` from both `0xD3` and `0x04` . Either the panel's SDO is not populated on this module or the read path is misconfigured. One negative does not separate them.
- **Phantom touches.** Two independent unattended runs logged spurious presses at ~374 s and ~390 s; in the first, they launched a program into slot 2. Real, reproducible, and unexplained.
- **Transmit still does not reach the air** (`UM-NATOS-028 §5`).

8. Numbers

Theories eliminated before the cause was found	11
Instruments caught lying, cumulative	15
Characters changed to fix it	1
Full-view blit, FIFO fallback	55.8 ms
Full-view blit, DMA working	31.4 ms

The Scheduler was right that the renderer was mid-transfer. He was wrong about which one, and so was everyone else.